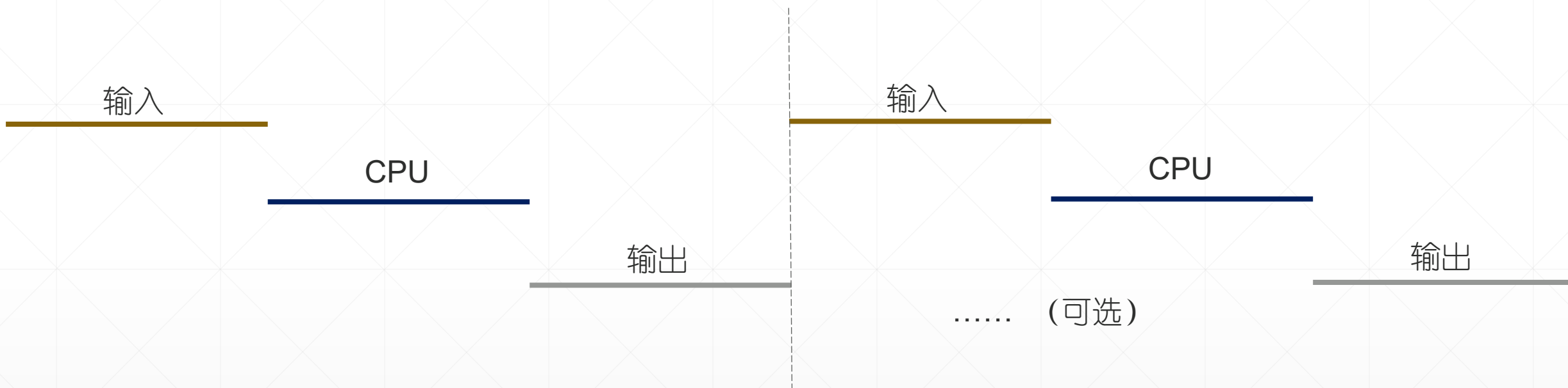


操作系统

第二章 进程管理

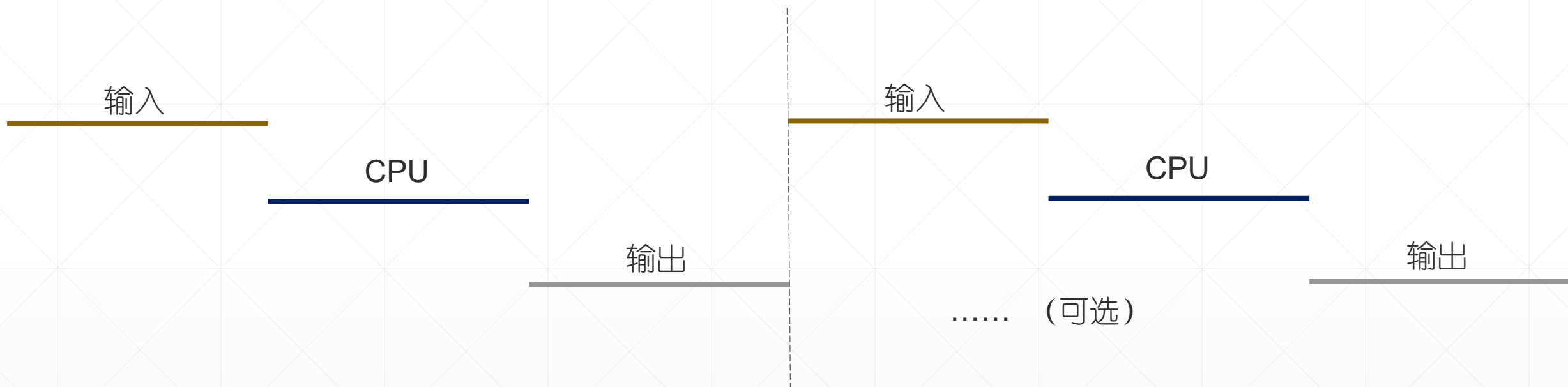
2、进程对资源的访问 & 进程的状态和管理

一、应用程序的 输入、计算 和 输出



应用程序。处理数据、做出决策。

- 输入阶段，为应用程序提供一批新数据；
- 计算阶段，**CPU**处理这批新数据；
- 输出阶段，运行结果 持久化（存盘）或 作用于物理世界。



输入设备 ?
计算阶段
输出设备 ?

输入阶段：输入设备忙，**CPU**空闲，输出设备空闲
计算阶段：**CPU**忙，输入、输出设备空闲
输出阶段：输出忙，输入设备、**CPU**空闲

使用的硬件



例：应用程序的 输入、输出阶段

```
int main( int argc, char* argv[] )
{
    if(argc!=3)
        printf("Usage: copy oldfile newfile\n");

    int oldFile = open(argv[1],O_RDONLY);
    int newFile = open(argv[2],O_WRONLY|O_CREAT,S_IRUSR|S_IWUSR);

    char c;
    while( read(oldFile,&c,1) == 1 )
        write(newFile,&c,1);

    printf("Copy Done\n");

    close(oldFile);
    close(newFile);
}
```

二、细节：输入设备控制

- 键盘, scanf、cin . . .
 - 等待用户按键
 - 键盘、CPU读键盘输入（将用户输入的ASCII字符写入内存指定区域）
 - 读内存变量、获取系统准备的键盘输入
- 磁盘, read磁盘文件
 - 向磁盘发读命令, 等待数据
 - 磁盘、DMA控制器读磁盘数据（将文件数据块写入内存）
 - 读内存中的文件数据块、获取DMA控制器送来的磁盘文件数据

细节：输出设备控制

- 屏幕, printf、cout。。。
-
- 磁盘, write磁盘文件
 - CPU, 内存准备需要存盘的数据块副本
 - CPU 向 DMA 和 磁盘发写命令: “将内存首地址***, 尺寸^^^的数据块写入磁盘, 起始扇区号###”
 - 磁盘、DMA控制器写磁盘数据 (内存中的磁盘数据块写回磁盘)
 - over

磁盘文件, 异步写

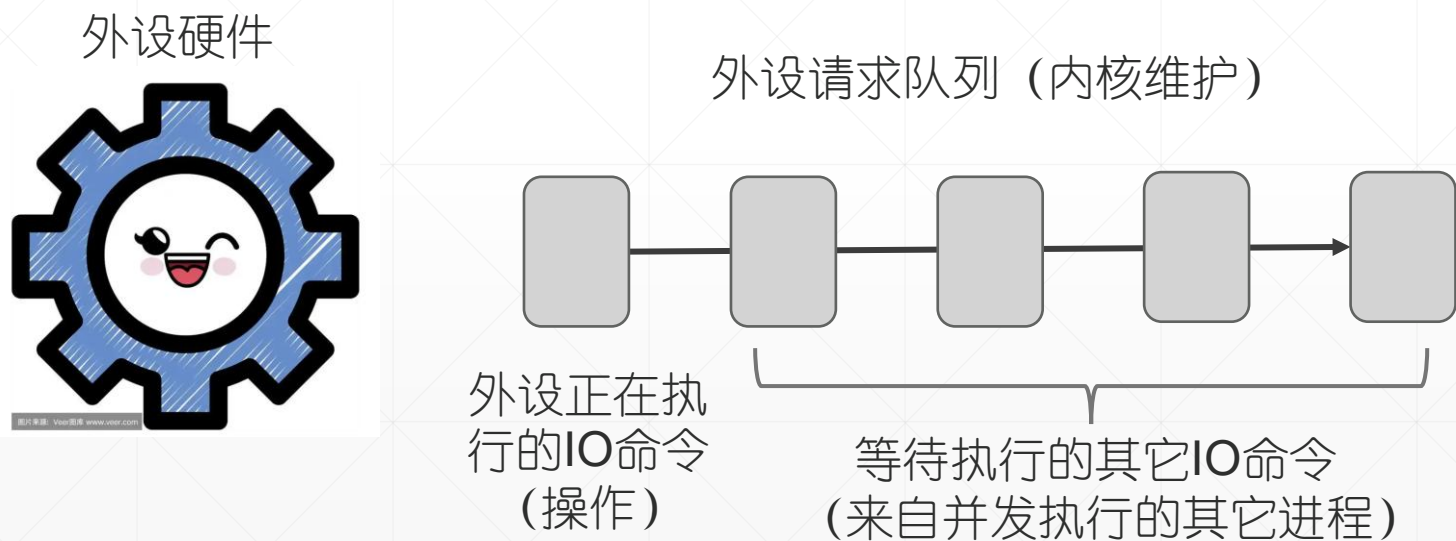
CPU无需等待。下个操作几乎100%成功。。。数据落盘, CPU无需再做任何事

读操作是同步的，因为CPU等待处理输入的新数据

- 键盘，scanf、cin。。。
 - 等待用户按键
 - 键盘、CPU读键盘输入（将用户输入的ASCII字符写入内存指定区域）
 - 读内存变量、获取系统准备的键盘输入
- 磁盘，read磁盘文件
 - 向磁盘发读命令，等待数据
 - 磁盘、DMA控制器读磁盘数据（将文件数据块写入内存）
 - 读内存中的文件数据块、获取DMA控制器送来的磁盘文件数据

外设硬件的资源使用模型

- 外设（单台），每次只能接纳CPU发来的一条命令。全程执行完毕，数据就位之前，不能向它发下条命令。所以，每个外设是一台互斥资源，最简单地可以将其建模为单队列服务系统。



三、单道 和 多道

- 单道：单道系统一次只能跑1个程序，这个程序运行完毕，系统才能装入、运行另一个程序。时序图（甘特图）



- 多道：多道系统的内存中存放着多个程序。程序IO时，CPU可以执行其它程序。

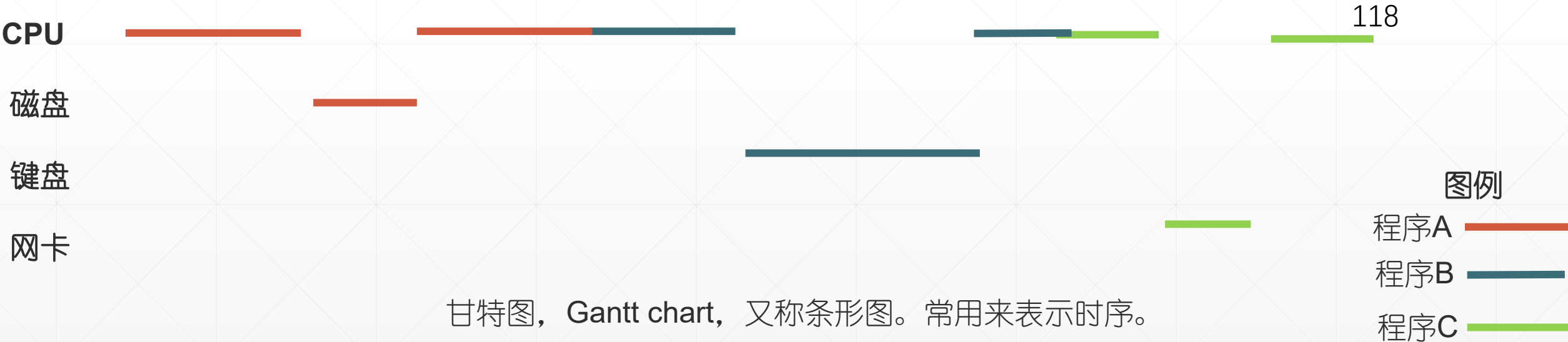


多道系统的好处在于：CPU和外设可以并行操作，提高资源的利用率。



例1：某单道系统， 需要执行3个程序。先来先服务。
画时序图。计算完成所有计算任务的总耗时（118ms）。

程序	进入系统的时刻	第一轮计算时间	外设IO时间 (使用不同的外设)	第二轮计算时间 (之后结束)
A	0	20	10 (读磁盘)	20
B	10	15	20 (读键盘)	10
C	20	5	8 (读网卡)	10



甘特图， Gantt chart， 又称条形图。常用来表示时序。

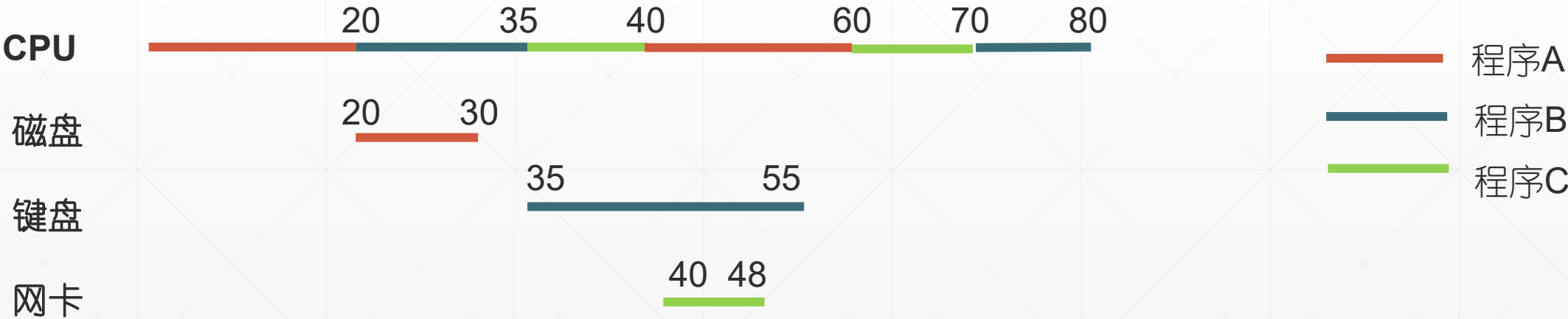


规则： 启动IO，程序放弃CPU。
IO完成后，程序可以使用CPU继续运行

例2：多道程序同时运行（先来先服务）

■ 80ms

程序	进入系统的时间	第一轮计算时间	外设IO时间	第二轮计算时间（之后结束）
A	0	20	10（读磁盘）	20
B	10	15	20（读键盘）	10
C	20	5	8（读网卡）	10



总结

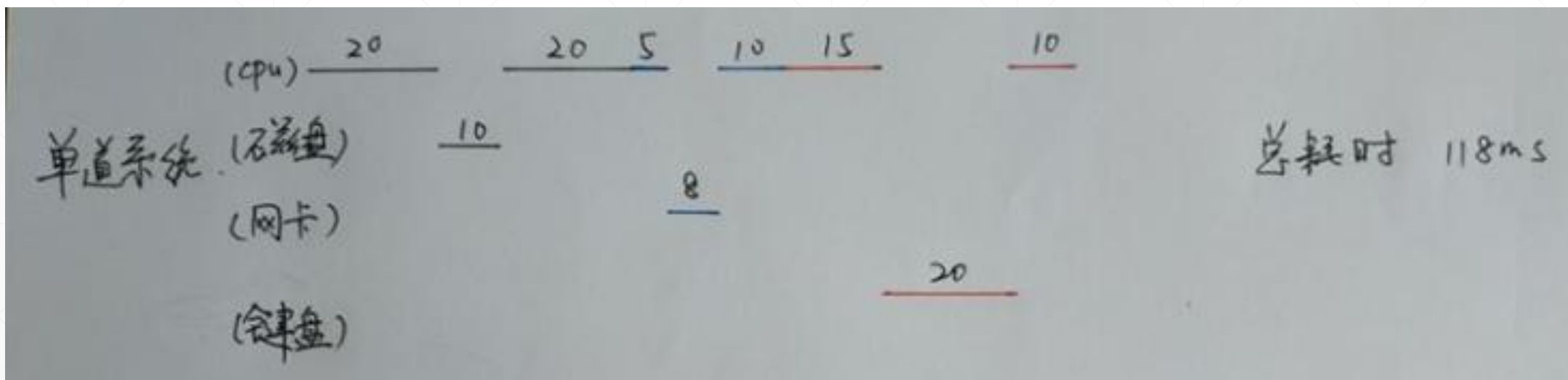
- 单道系统的缺点：↵
 - 无论何时，至多一个设备在工作。IO、计算不能并行。↵
 - 系统资源利用率低。吞吐率低。↵
 - 单道系统的优点：↵
 - 程序独占计算机系统。从开始运行至运行结束，耗时最短。↵
 - 多道系统的优点：↵
 - 同时为很多用户提供服务。↵
 - 系统（所有资源）利用率高，吞吐率高。↵
- 优点，来自计算、IO 并行的设计。↵

最后。内存中存放好多程序供 CPU 选择是有好处的。emm... CPU 空闲时，找到没在 IO 可供执行的程序，概率高。有利于保持 CPU 忙碌。另一方面，忙碌的 CPU 会执行更多的应用程序，发出更多的 IO 命令，从而保持 IO 设备忙碌。↵

内存装的程序越多，越有可能让所有设备保持忙碌，从而提高系统的吞吐量。这里有一个度，系统中并发的进程总数要控制，我们未来展开。↵

例 3 单道方式运行。采用优先级高者先执行的调度策略。优先级： 程序A<程序B<程序C。

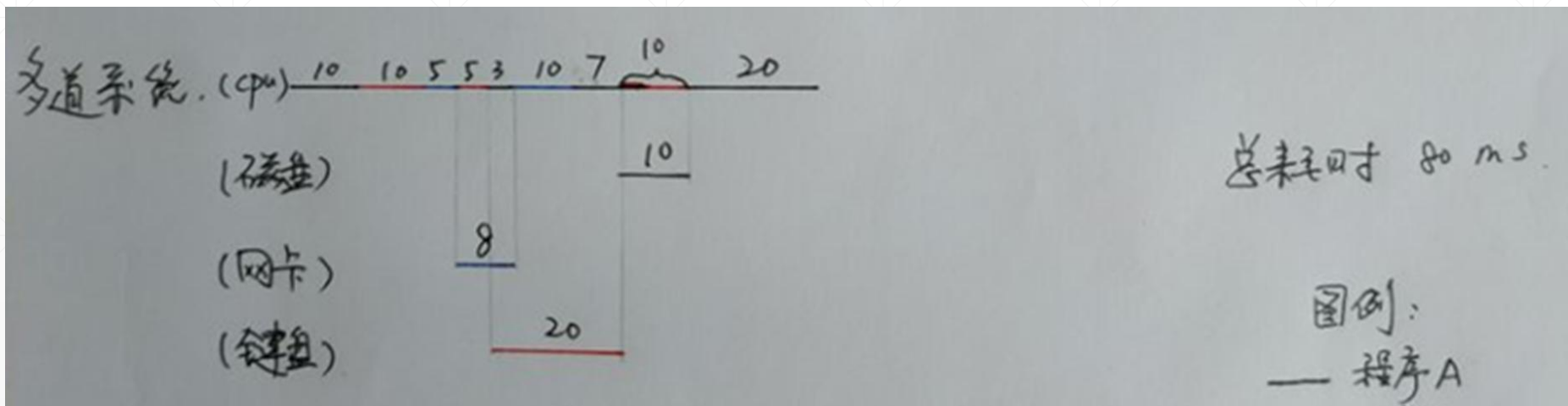
程序	进入系统的时间	第一轮计算时间	外设IO时间	第二轮计算时间（之后结束）
A	0	20	10（读磁盘）	20
B	10	15	20（读键盘）	10
C	20	5	8（读网卡）	10



例 4

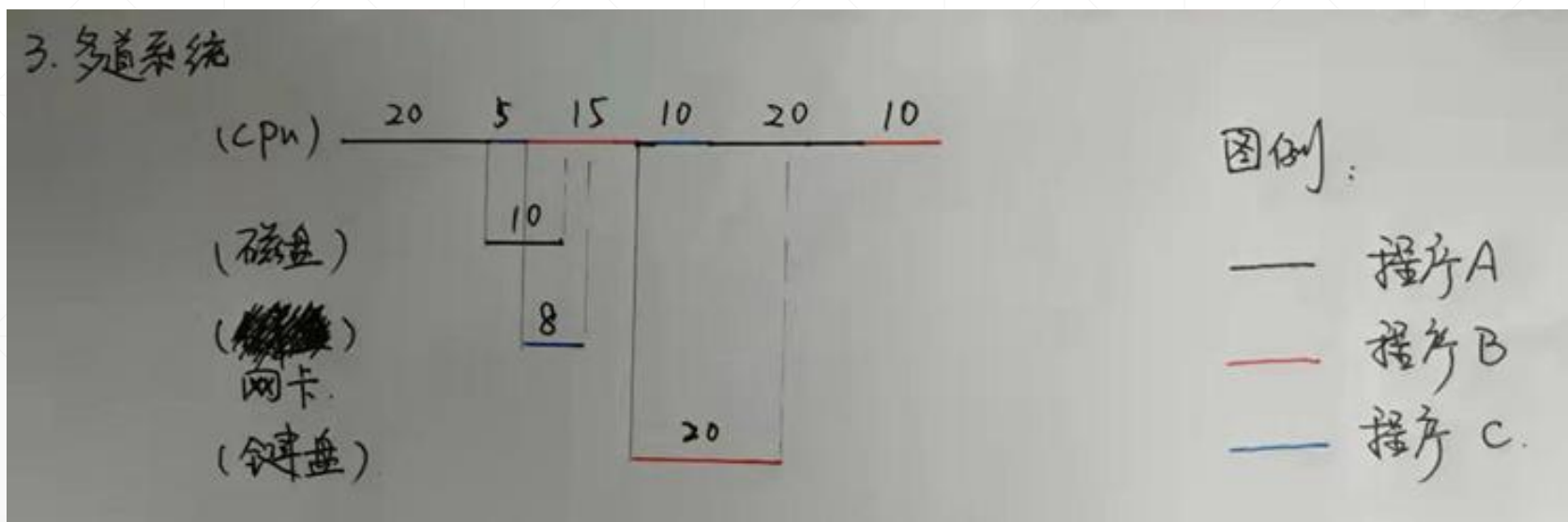
多道方式运行。采用优先级高者先执行的调度策略。优先级： 程序A<程序B<程序C 。如果出现优先级更高的程序，正在执行的程序让出CPU。

程序	进入系统的时间	第一轮计算时间	外设IO时间	第二轮计算时间（之后结束）
A	0	20	10（读磁盘）	20
B	10	15	20（读键盘）	10
C	20	5	8（读网卡）	10



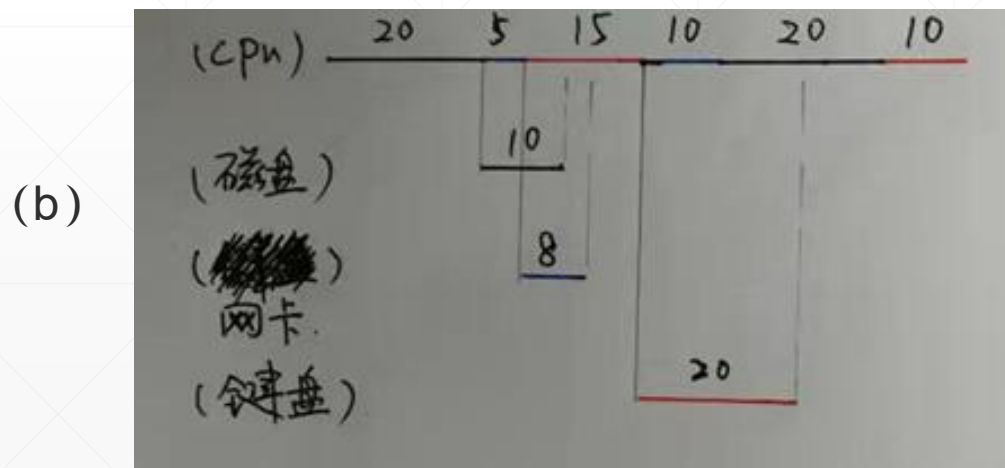
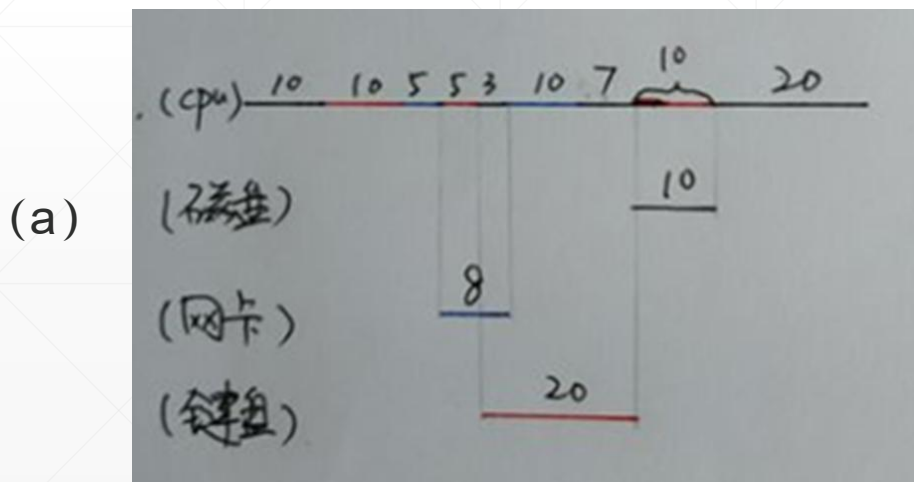
例 5 多道方式运行。采用优先级高者先执行的调度策略。优先级： 程序A<程序B<程序C。
出现优先级更高的程序，正在执行的程序不会让出CPU，会一直运行直至IO或运行结束。

程序	进入系统的时间	第一轮计算时间	外设IO时间	第二轮计算时间（之后结束）
A	0	20	10（读磁盘）	20
B	10	15	20（读键盘）	10
C	20	5	8（读网卡）	10



可剥夺的调度策略 (a) 和 不可剥夺的调度策略 (b)

程序	进入系统的时间	第一轮计算时间	外设IO时间	第二轮计算时间 (之后结束)
A	0	20	10 (读磁盘)	20
B	10	15	20 (读键盘)	10
C	20	5	8 (读网卡)	10





结论

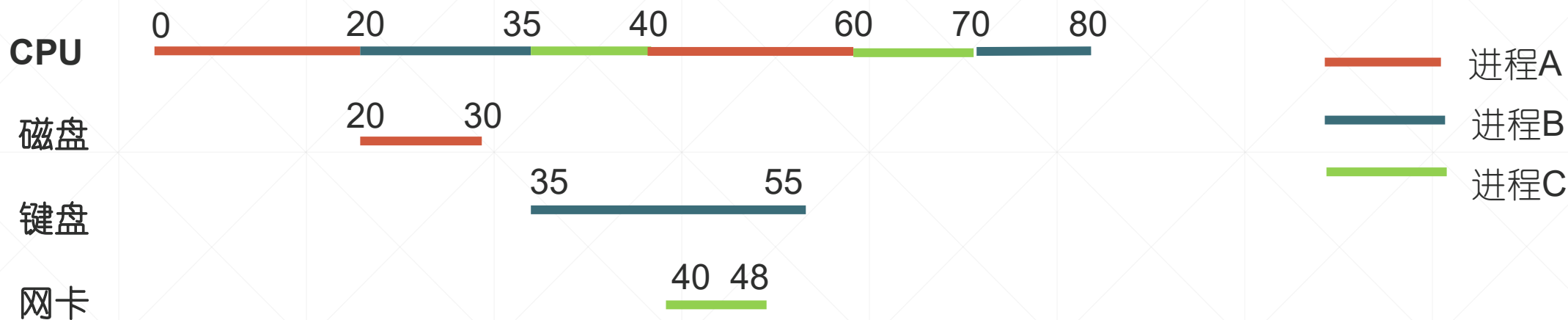
- 多道系统，输入、输出设备 和 CPU可以同时忙碌。所以，资源利用率高、应用程序平均周转时间短。



四、进程的调度状态

- 多道系统，资源为所有并发进程共享。得到资源，进程前进；未得到资源，进程暂停。
- 运行态，正在使用CPU
- 就绪态，等待使用CPU
- 阻塞态（Unix，睡眠态），不能使用CPU。

例:



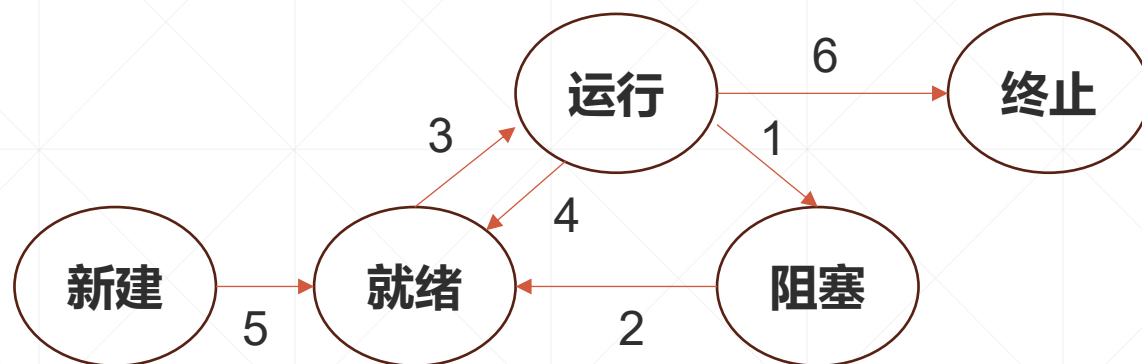
习题1

进程A的状态: [0, 20]运行, [20, 30]阻塞, [30, 40]就绪, [40, 60]运行
 进程B的状态:
 进程C的状态:

习题2: 单核系统, 并发10个进程。运行态进程最多几个, 最少几个; 就绪态进程~~~; 阻塞态进程~~~

习题3: 8核系统, 并发10个进程。。。。

进程状态转换图 (可剥夺系统)



- 1、入睡（阻塞）：进程无法前进主动放弃CPU
- 2、唤醒：IO完成.....进程恢复可执行状态。
- 3、被调度：被选中成为现运行进程。
- 4、被抢占：系统中出现优先级更高的进程。
- 5、进程图像创建完毕，就绪态
- 6、应用程序执行完毕或被杀死

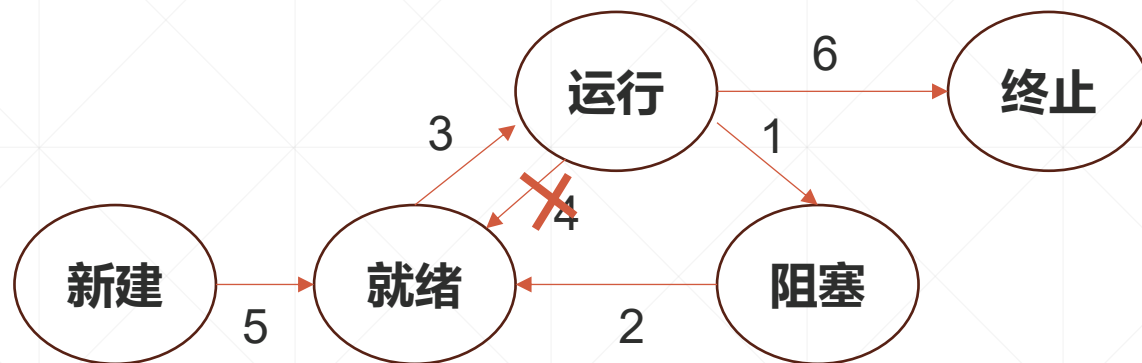
入睡原因

- 1) 启动IO操作
- 2) sleep(5), 启动定时器
- 3) 内存中没有新数据
- 4) 进程访问的数据被其它进程锁住了
- 5) 排队等待资源

唤醒事件

- 1) IO完成
- 2) 定时器到期, 5s后进程恢复运行
- 3) 新数据就绪 (前驱任务完成, IO完成也算)
- 4) 对方数据访问完毕, 解锁
- 5) 获得使用权

进程状态转换图 (不可剥夺系统)



- 1、入睡（阻塞）：进程无法前进主动放弃CPU
- 2、唤醒：IO完成.....进程恢复可执行状态。
- 3、被调度：被选中成为现运行进程。
- ~~4、被抢占：系统中出现优先级更高的进程。~~
- 5、进程图像创建完毕，就绪态
- 6、应用程序执行完毕或被杀死

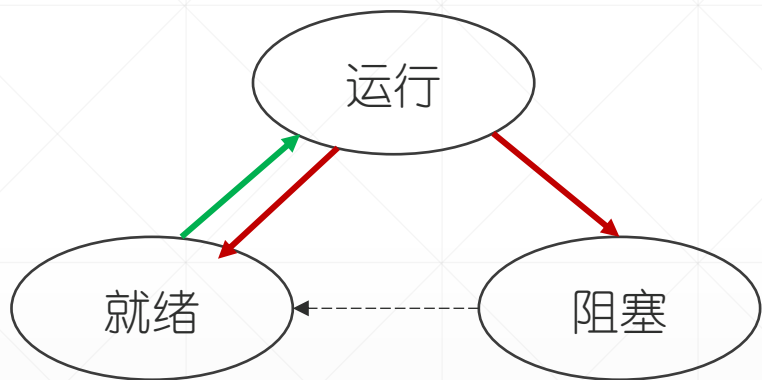
入睡原因

- 1) 启动IO操作
- 2) sleep(5), 启动定时器
- 3) 内存中没有新数据
- 4) 进程访问的数据被其它进程锁住了
- 5) 排队等待资源

唤醒事件

- 1) IO完成
- 2) 定时器到期, 5s后进程恢复运行
- 3) 新数据就绪 (前驱任务完成, IO完成也算)
- 4) 对方数据访问完毕, 解锁
- 5) 获得使用权

进程切换 Switch



1、现运行进程放弃CPU

- 保护CPU现场（通用寄存器，尤其是EIP、ESP&EBP，前者是日后恢复运行下条指令的虚地址，后者是栈顶帧位置）
- 放弃CPU

2、遍历就绪队列，挑选优先级最高的就绪进程（新选中进程）

3、恢复新选中进程的现场

- 为MMU建立该进程的地址映射关系
- 恢复CPU现场（通用寄存器，尤其是EIP、ESP&EBP）



（成了，CPU进入执行新选中进程的模式）

证据

取指

CPU执行单元 → EIP → MMU → 新选中进程下条指令的物理地址 → 内存

新选中进程的
地址映射关系

新选中进程下条指令

访问局部变量

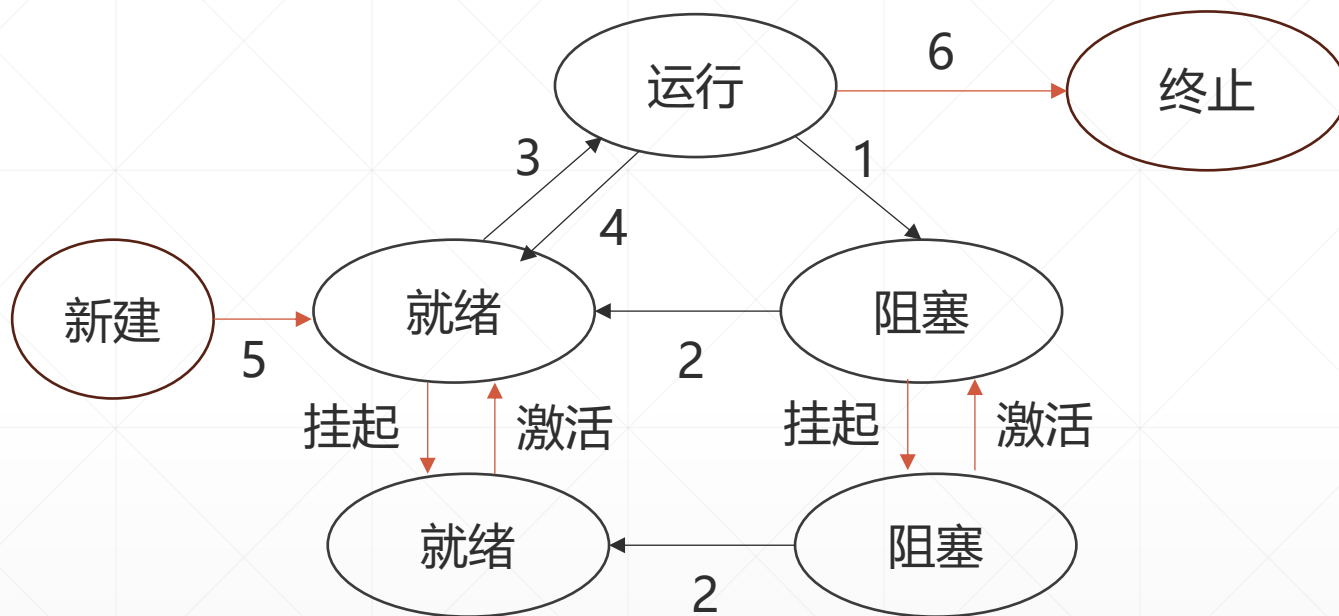
CPU执行单元 → [EBP-***] → MMU → 新选中进程, 局部变量的物理地址 → 内存

新选中进程的
地址映射关系

局部变量的值

.....

挂起 (suspend) 和 激活 (activate)



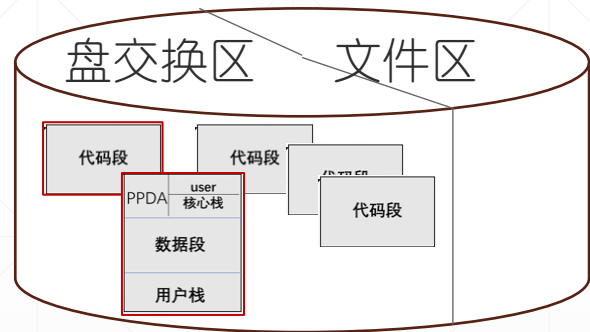
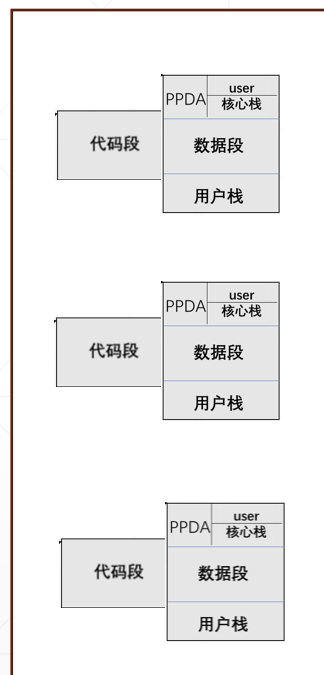
- 挂起，暂停执行。被debug的程序走到断点
- 激活，恢复执行。step into, step over, resume

换入 (swap in) 、换出 (swap out)

主存 (内存)

- 内存紧张时，系统会将部分进程图像放在盘交换区。待内存有空，恢复执行。
- 如图，Unix V6++系统，4个并发进程，3个进程在内存（代码段、可交换部分全部在内存），1个进程在盘交换区（红框）。

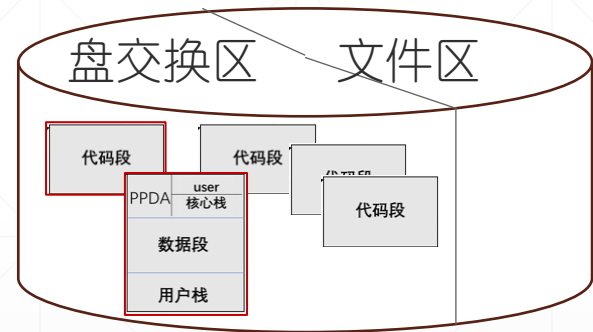
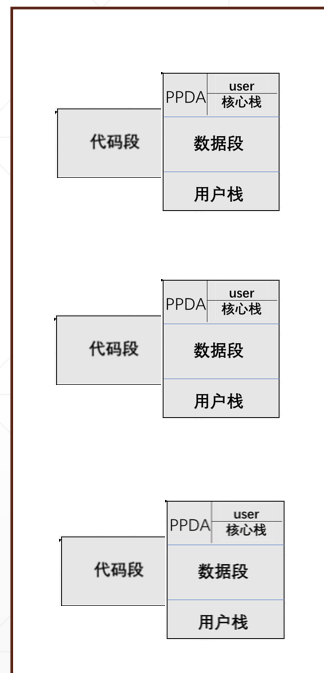
备注：盘交换区是磁盘上专门用来放进程图像的区域。
Unix V6++，2#扇区~199#扇区。



图：Unix V6++系统的存储介质
主存 + 盘交换区 放进程图像，文件区 放 磁盘文件

Unix V6++对盘交换区的使用、换入换出操作

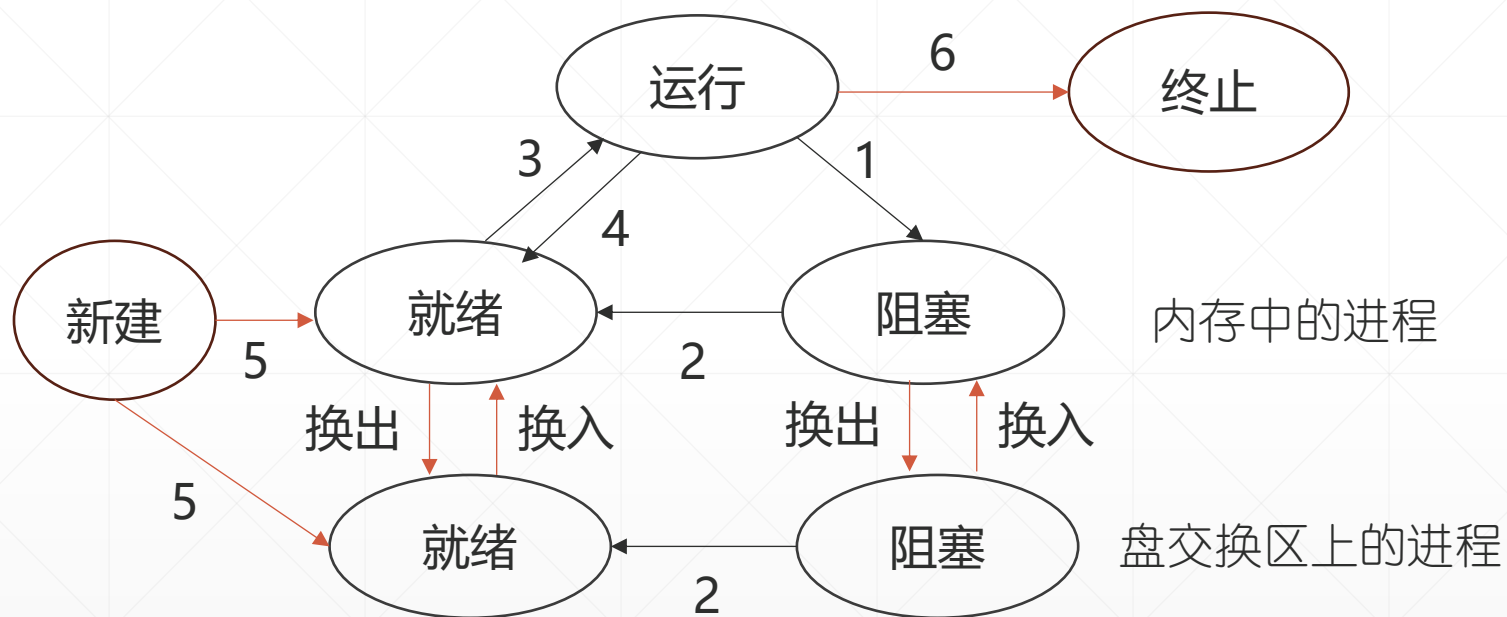
主存（内存）



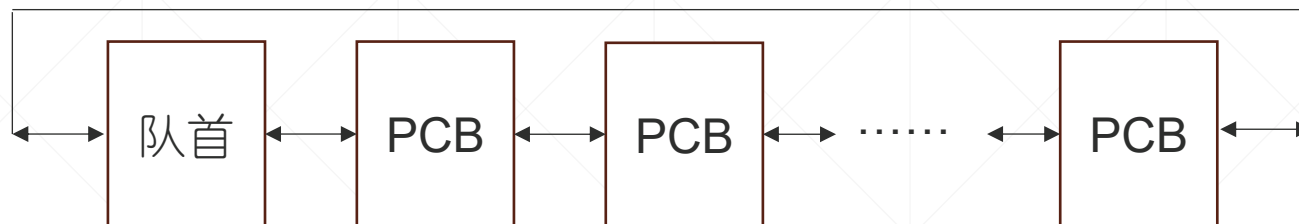
图：Unix V6++系统的存储介质
主存 + 盘交换区 放进程图像，文件区 放 磁盘文件

- 盘交换区上的进程没有执行权限（因为user结构不在内存）。
- 换出（进程PA）：
 - 盘交换区分配连续存储空间，将PA的可交换部分复制到磁盘，释放内存。
 - 递减正文段引用计数（Text结构，x_ccount--），为0，内存中没有进程引用这个正文段，释放内存。
- 换入（进程PA）：
 - 分配内存，复制PA的可交换部分，释放盘交换区。
 - 递增正文段引用计数（Text结构，x_ccount++），为1，分配内存，复制PA的代码段。

盘交换区的使用，状态转换图



五、进程管理，进程队列（单核系统）



- 管理可执行进程（就绪、现运行），用就绪进程队列。
 - 系统只有一根就绪进程队列。调度时，从就绪队列中挑选新运行进程。
- 管理阻塞进程，用阻塞进程队列。
 - 磁盘的IO请求队列，排着等待执行IO操作，或者等待IO操作完成的进程。
 - 信号量队列，排着等待访问共享资源的进程。
 - 时钟队列，排着等待定时器到期的进程。
 -有多少种入睡理由，就有多少根阻塞队列。

进程管理，进程队列（多核系统）

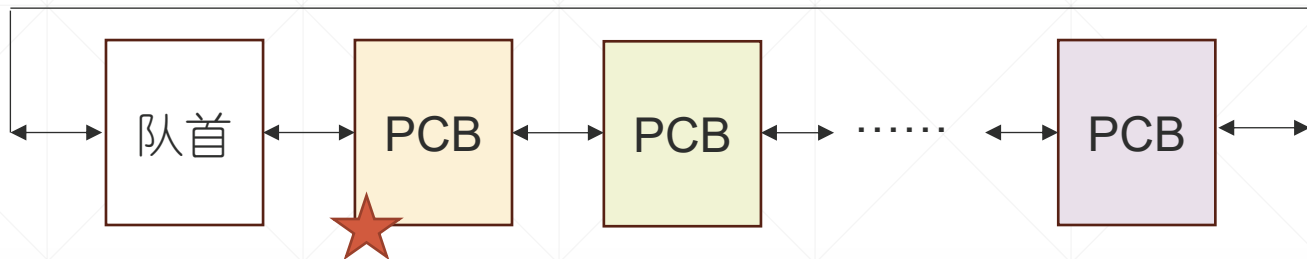


- 管理可执行进程（就绪、现运行），用就绪进程队列。优化的实现是：
 - **每颗CPU一根就绪进程队列**。现运行进程放弃CPU时，从自己的就绪队列中挑选新运行进程。
 - 每个就绪进程队列，用复杂的结构提升调度算法的执行效率。
- 管理阻塞进程，用阻塞进程队列。阻塞队列，所有CPU共享。
 - 磁盘的IO请求队列，排着等待执行IO操作，或者等待IO操作完成的进程。
 - 信号量队列，排着等待访问共享资源的进程。
 - 时钟队列，排着等待定时器到期的进程。
 -有多少种入睡理由，就有多少根阻塞队列。

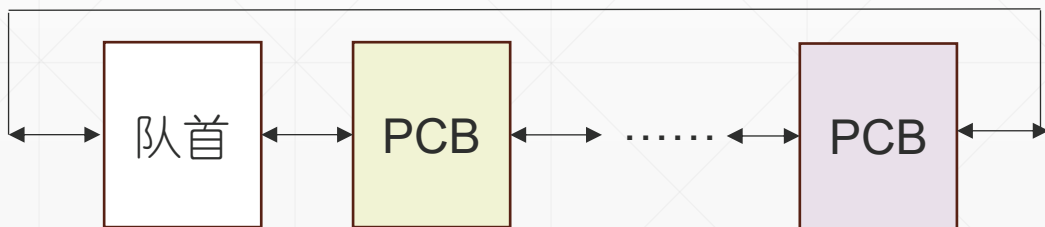
进程状态转换涉及的队列操作

1、入睡

就绪队列，队首是现运行进程

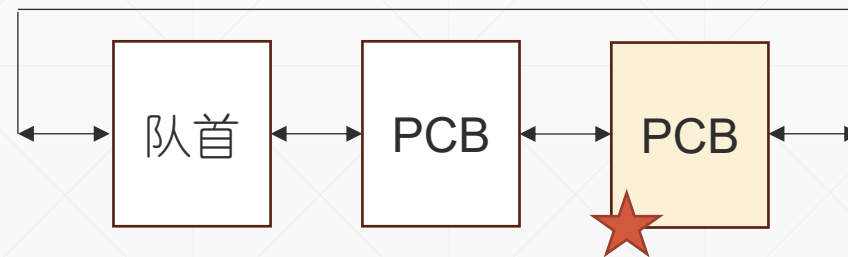


就绪队列



- 1) 从就绪队列摘除现运行进程PCB (黄)
- 2) 修改调度状态为阻塞
- 3) 插入阻塞队列(队尾, 未必。。。)
- 4) **Swch()**放弃CPU, 黄色进程阻塞, 紫色进程成为新运行进程

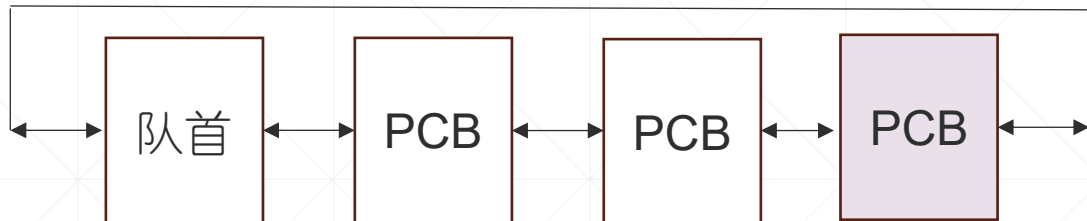
阻塞队列



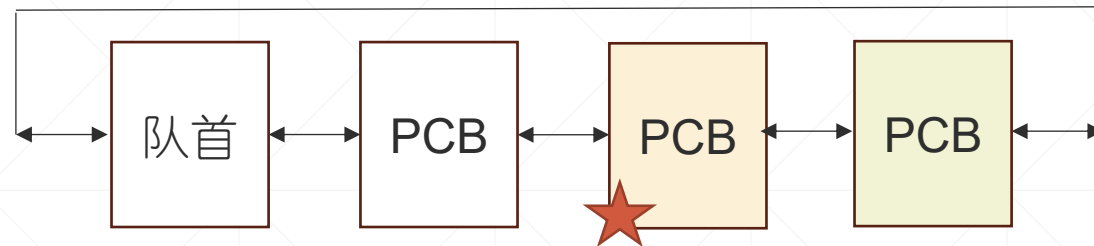
进程状态转换涉及的队列操作

2、唤醒（黄色进程）

就绪队列

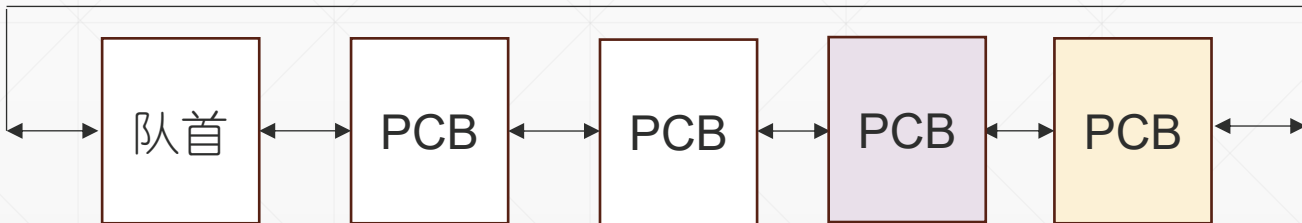


阻塞队列

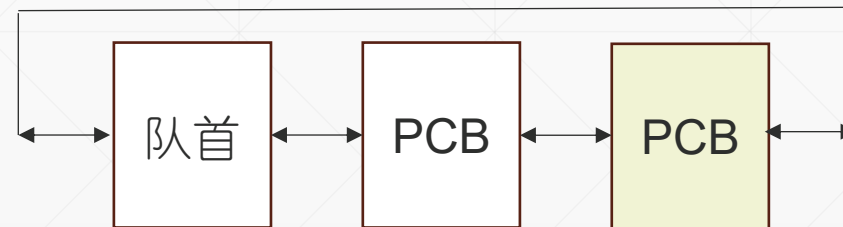


- 1) 从阻塞队列摘除需要唤醒的进程PCB（黄）
- 2) 修改调度状态为就绪
- 3) 插入就绪队列(队尾，未必。。。)
- 4) 可剥夺系统
if (被唤醒进程优先级更高)
置剥夺标识

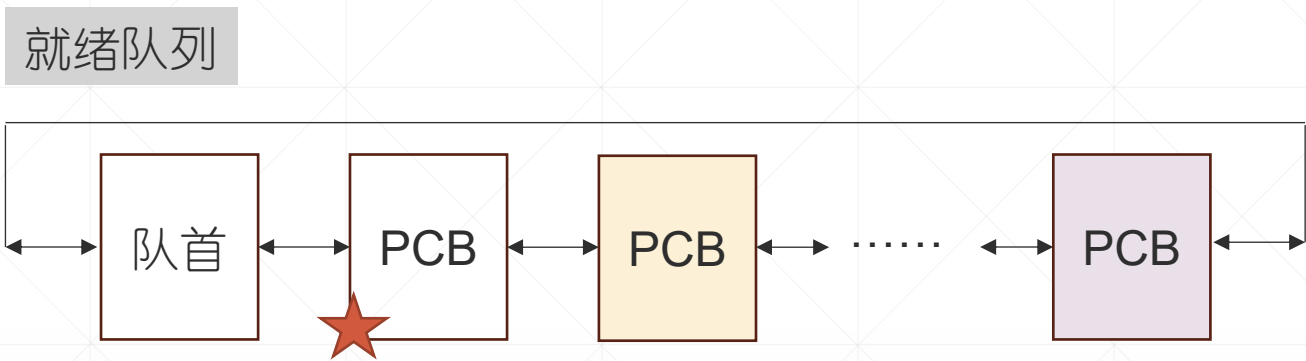
就绪队列



阻塞队列



进程状态转换涉及的队列操作



3、现运行进程被剥夺

在代码固定的位置（调度点），检测剥夺标识

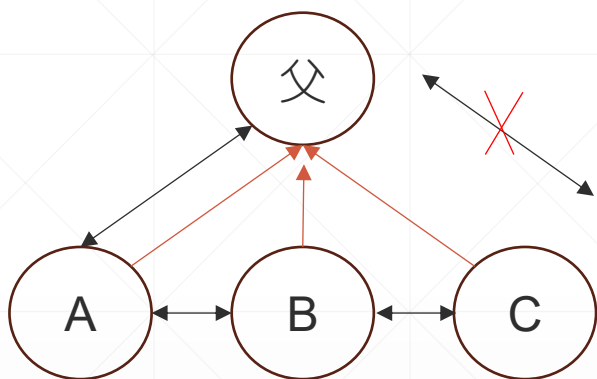
if(存在优先级更高的就绪进程)
Switch()放弃CPU

现运行进程就绪，另一个进程现运行。

4、创建新进程，入就绪队列

5、进程终止，出就绪队列

六、进程管理，进程的家族关系



子进程(PCB), `parent`指针指向父进程(PCB)

父进程的`child`指针指向第一个子进程。第一个子进程的`sibling`指向第二个子进程 最后一个子进程的`sibling`为空。

最后一个子进程的`sibling`也可以指向父进程。这样，从父进程的`child`出发，我们就得到了一个完整的双向队列。

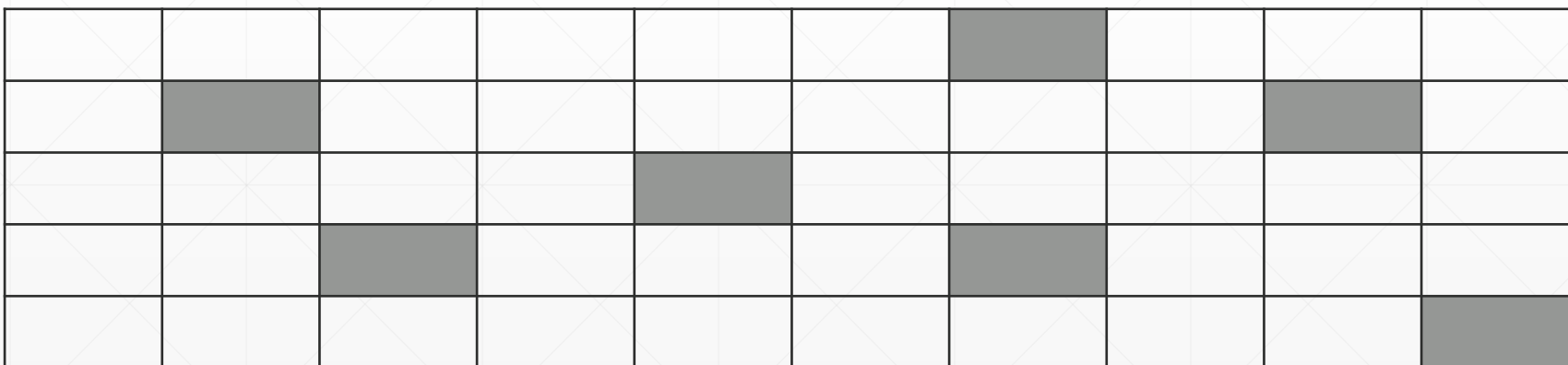
可以用 `sibling==父进程PCB` 作为最后一个子进程的判断条件。这样做，可能会有好处：比如新进程插队尾，会很方便。系统不用遍历链表，直接定位尾部，插入操作的时间复杂度是 $O(1)$ 。不一定真的有用，看系统功能需求。

七、PCB存放的空间

- Unix V6++，静态数组 Process



- Linux, Slab。Slab是尺寸固定的连续内存块，划分成多个等长的小块，每个小块可以放一个PCB。Slab按需分配，系统用 task_struct cache 管理分配给 task_struct 的所有Slab。创建新进程的时候，new一个task_struct。如果没有空闲小块，向系统申请一个新的Slab。进程终止时，释放task_struct，原先占用的小块变空闲，可以分配给新进程。若出现空闲的slab，回收其占用的物理内存



八、PID的管理

- 为新进程分配的PID一定要唯一的。

Unix V6++，怎样分配PID？ 创建子进程时，会执行clone函数。为子进程分配pid是其中的一个子步骤

- Linux。pid 位图。

```
ProcessManager.cpp Process.cpp X
273 }
274
275 void Process::Clone(Process& proc)
276 {
277     User& u = Kernel::Instance().GetUser();
278
279     /* 拷贝父进程Process结构中的大部分数据 */
280     proc.p_size = this->p_size;
281     proc.p_stat = Process::SRUN;
282     proc.p_flag = Process::SLOAD;
283     proc.p_uid = this->p_uid;
284     proc.p_ttyp = this->p_ttyp;
285     proc.p_nice = this->p_nice;
286     proc.p_textp = this->p_textp;
287
288     /* 建立父子关系 */
289     proc.p_pid = ProcessManager::NextUniquePid();
290     proc.p_ppid = this->p_pid;
```

```
ProcessManager.cpp X
55
56
57 unsigned int ProcessManager::NextUniquePid()
58 {
59     return ProcessManager::m_NextUniquePid++;
60 }
```

九、用 pid 定位 PCB

- Unix V6++，遍历静态数组 Process，找那个pid

pid	pid		pid			pid			pid
-----	-----	--	-----	--	--	-----	--	--	-----

- Linux，有一个pid hash。 定位时，用pid搜索这张表

						pid			
	pid							pid	
				pid					
		pid				pid			
									pid